Unified Deadline Management System (DeadlineHub)

Project Report

---

1. Title of the Project

DeadlineHub — A Unified Deadline Management System for Students and Interns *Built on the MERN Stack (MongoDB, Express, React, Node.js)*

---

2. Group Details

Field	Details
Group Number	23
Project Title	DeadlineHub — Unified Deadline Management System
Submission Date	10th April 2026

#	Enrollment Number	Student Name
1	230953466	Saksham Khetan
2	230953330	Mridul Arora

---

3. Project Abstract

Students and interns routinely juggle deadlines scattered across college portals, Microsoft Teams, internship trackers, and personal reminders. This fragmentation leads to missed submissions, poor prioritization, and constant context-switching between platforms.

DeadlineHub is a full-stack web application built on the **MERN stack** that consolidates all deadlines into a single, intelligent dashboard. Users can create and track tasks tagged by source (College, Internship, Personal), view them on a unified calendar with month/week/day views, receive visual urgency cues (Overdue, Critical, Approaching, Safe), and analyze their productivity through charts. The application also models Microsoft Teams meetings and Planner items, and provides a configurable reminder system.

Core functionalities: JWT-based user authentication; full CRUD on tasks, meetings, and planner items; smart urgency classification; search, filter, and sort; productivity analytics; browser notifications; responsive UI.

Technologies used:

- **Frontend:** React 19, Vite, React Router v7, React-Bootstrap, react-big-calendar, Recharts, date-fns, React Context + useReducer
- **Backend:** Node.js, Express 4, Mongoose, JWT (jsonwebtoken), bcryptjs, CORS, Morgan, dotenv
- **Database:** MongoDB (document store for users, tasks, meetings, planner items)
- **Tooling:** npm, nodemon, ESLint, Git

---

4. High-Level Design

4.1 System Architecture

```
+-------------------------------------------------------------+
|                     Browser (Client)                        |
|  React 19 + React Router + Context API + Bootstrap UI        |
+------------------------------^------------------------------+
                               | HTTPS / JSON
                               | Authorization: Bearer <JWT>
+------------------------------v------------------------------+
|                Express REST API (Node.js)                   |
|  Routes -> Controllers -> Mongoose Models                   |
|  Middleware: CORS, JSON parser, Morgan, JWT auth, errors    |
+------------------------------^------------------------------+
                               | Mongoose ODM
+------------------------------v------------------------------+
|                    MongoDB Database                         |
|    Collections: users, tasks, meetings, planneritems        |
+-------------------------------------------------------------+
```

4.2 Database Architecture (ER-style overview)

```
  users                          tasks
 +------+                    +-------------+
 | _id  |<-------owner-------| user (ref)  |
 | name |                    | title       |
 | email|                    | description |
 | pwd  |                    | source      |
 +------+                    | deadline    |
    |                        | priority    |
    |                        | completed   |
    |                        | link        |
    |                        +-------------+
    |
    |        meetings                 planneritems
    |      +-------------+            +-------------+
    +----->| user (ref)  |     +----->| user (ref)  |
    |      | title       |     |      | title       |
    |      | organizer   |     |      | bucket      |
    |      | time        |     |      | assignedTo  |
    |      | duration    |     |      | dueDate     |
    |      | recurring   |     |      +-------------+
    |      | channel     |     |
    |      +-------------+     |
    +--------------------------+
```

Each non-user document carries a `user` ObjectId reference, enforcing per-user data isolation on every query.

4.3 REST API Surface

Module	Method & Endpoint	Purpose
Auth	`POST /api/auth/register`	Create account, returns JWT
Auth	`POST /api/auth/login`	Authenticate, returns JWT
Auth	`GET /api/auth/me`	Current user profile
Tasks	`GET /api/tasks`	List tasks (filter by source, priority, status)
Tasks	`POST /api/tasks`	Create task
Tasks	`GET /api/tasks/:id`	Fetch single task
Tasks	`PUT /api/tasks/:id`	Update task
Tasks	`DELETE /api/tasks/:id`	Delete task
Tasks	`PATCH /api/tasks/:id/toggle`	Toggle completion
Meetings	`GET/POST/PUT/DELETE /api/meetings`	CRUD for Teams meetings
Planner	`GET/POST/PUT/DELETE /api/planner`	CRUD for planner items
Analytics	`GET /api/analytics`	Aggregated KPIs and distributions

All non-auth endpoints are protected by JWT middleware and scoped to `req.user._id`.

4.4 Module Breakdown

Frontend modules: Layout (Navbar, Sidebar), Calendar Dashboard, Task Manager, Analytics Dashboard, Teams Integration, Reminders, Global State (TaskContext).

Backend modules: `config/db.js` (Mongo connection), `models/` (User, Task, Meeting, PlannerItem), `controllers/` (business logic), `routes/` (Express routers), `middleware/` (auth + error handling), `server.js` (app composition).

---

5. Components / Concepts Used

Component / Concept	Layer	Purpose in the Application
React 19	Frontend	Component-based UI rendering and hooks-based state
Vite	Frontend	Fast dev server and production bundler
React Router v7	Frontend	Client-side routing across dashboard pages
React Context + useReducer	Frontend	Global task state with predictable actions
React-Bootstrap	Frontend	Responsive grid, modals, forms, cards
react-big-calendar	Frontend	Month / week / day / agenda calendar views

Recharts	Frontend	Bar, pie, and area charts for analytics
date-fns	Frontend	Date arithmetic and formatting
React Icons	Frontend	Iconography across UI
Node.js + Express 4	Backend	HTTP server and REST routing
Mongoose ODM	Backend	Schema modeling and MongoDB access
MongoDB	Database	Document store for all collections
JWT (jsonwebtoken)	Backend	Stateless authentication tokens
bcryptjs	Backend	One-way password hashing with salt
CORS middleware	Backend	Controlled cross-origin access for the React client
Morgan	Backend	HTTP request logging
dotenv	Backend	Environment variable management
express-async-handler	Backend	Clean async error propagation
REST API principles	Architecture	Resource-oriented endpoints and verbs
MVC pattern	Architecture	Models, controllers, and routes separation
ESLint	Tooling	Code quality and consistency
Git	Tooling	Version control

---

6. Key Screens and Descriptions

The working application consists of the following principal screens, each bound to one or more backend endpoints:

1. **Login / Register Screen** — JWT-backed authentication form. Users create an account or sign in; the Express server issues a JWT that the React client stores and attaches as a `Bearer` token on all subsequent API calls.

2. **Calendar Dashboard (Home)** — A unified month / week / day / agenda calendar built with react-big-calendar, displaying deadlines from every source color-coded by urgency (Overdue, Critical, Approaching, Safe). Summary KPI cards at the top show counts for Pending, Critical, Approaching, and Safe tasks. Clicking any event opens a Task Quick View modal with full details and a "Mark Complete" action.

3. **Task Manager Page** — Grid of task cards with a real-time search bar, filters for source / priority / status, and sorting options (deadline, priority, title). Each card shows title, description, deadline, and badges for source, priority, and urgency, along with complete / edit / delete buttons. The "Add Task" button opens the Task Form modal, which maps to `POST /api/tasks` for creation and `PUT /api/tasks/:id` for edits.

4. **Analytics Dashboard** — Productivity insights aggregated from `GET /api/analytics`. Displays four KPI cards (Total, Completed, Missed, Completion Rate) and four Recharts visualizations: a

weekly activity bar chart, a source distribution donut, a priority distribution pie, and a monthly trend area chart.

5. **Microsoft Teams Integration Page** — A two-state page. When disconnected, it shows a Microsoft sign-in prompt; when connected, it lists synced meetings (title, organizer, time, duration, channel, recurring badge) and planner items grouped by bucket, served from `/api/meetings` and `/api/planner`.
6. **Reminder Settings Page** — Configuration panel for browser notifications (with permission request) and email alerts, plus a default reminder-time selector (15 minutes to 1 day before). Includes an overdue alert banner and a list of upcoming reminders sorted by deadline.
7. **Notification Badge (Navbar)** — A bell icon in the top navbar showing a live count of overdue and critical tasks, with a dropdown listing the most urgent items so users can jump directly to them from any page.

---

7. Details of Individual Contributions

Member	Primary Responsibilities	Key Deliverables
Saksham Khetan (230953466)	Backend development, database design, and core task/calendar frontend	Express server and project scaffolding; MongoDB schema and Mongoose models (User, Task, Meeting, PlannerItem); JWT authentication middleware and bcrypt password hashing; auth and task REST controllers with filtering, sorting, and toggle-completion endpoints; React layout (TopNavbar, Sidebar), Calendar Dashboard using react-big-calendar with urgency color-coding, and the Task Manager CRUD UI with TaskForm modal; TaskContext global state via Context API + useReducer.
Mridul Arora (230953330)	Analytics, integrations, reminders, UI polish, and documentation	Meetings, Planner, and Analytics REST controllers with per-user aggregation pipeline; Recharts-based Analytics Dashboard (KPIs, weekly bar chart, source donut, priority pie, monthly trend); Microsoft Teams integration page (meetings + planner buckets); Reminders module with browser notifications and notification badge in the navbar; React-Bootstrap theming, responsive styling, and helper utilities (urgency/badge/format); API integration layer (fetch wrapper + auth token handling); end-to-end testing and final project report & documentation.

---

8. Conclusion

DeadlineHub successfully demonstrates a complete MERN-stack solution to a real productivity problem faced by students and interns. On the **backend**, an Express + MongoDB REST API provides secure, JWT-protected CRUD endpoints for users, tasks, meetings, planner items, and analytics, with clean separation between models, controllers, routes, and middleware. On the **frontend**, a React 19 single-page application consumes the API through a Context-based state layer and renders a unified calendar, searchable task manager, productivity dashboard, Teams integration view, and configurable reminders.

Outcomes achieved:

- A working full-stack application with persistent, per-user data in MongoDB.
- Secure authentication using hashed passwords and stateless JWTs.
- A responsive, component-driven UI that scales from desktop to mobile.
- Smart urgency classification and visual analytics that turn raw deadlines into actionable insight.
- A modular codebase that cleanly separates frontend and backend concerns, making the system easy to extend (email notifications, real Microsoft Graph integration, mobile client, etc.).

The project gave the team hands-on experience with RESTful API design, ODM modeling, authentication flows, React state management, data visualization, and full-stack deployment — validating the MERN stack as a productive choice for rapidly building data-driven web applications.